AQ-09 · attestqa

Open-Source Release and Publishing

Version 1.1 (for sign-off) · 2026-10-10 · Author: Vijay Javvadi

1. Build in the open from the first day

The attestqa repository is public from the first commit of the v1.0.0 month, not from the first release. Three reasons. An open-source tool that asks to be trusted with test results should show how it was built, not only what it became. A public history timestamps every decision and rule, which matters for the papers built on it. And the earliest users of a QA tool are usually other testers, who are more likely to find it while it is being made than after.

What is never published: `.env` files, tokens and keys, ledgers from runs against private or company systems, and anything from a target whose owner has not agreed to automated testing. The repository's `.gitignore`, secret scanning and the scrubber in the engine are three separate barriers against that.

2. What each release publishes

Where	What	How
GitHub Releases	Tag `vX.Y.Z`, release notes, the signed Windows installer, SHA-256 checksums, a software bill of materials (CycloneDX)	Built and uploaded by GitHub Actions when Vijay pushes the tag
npm	Each changed package (`attestqa`, `@attestqa/*`), with provenance attached	Published by GitHub Actions through npm trusted publishing; no npm token is kept on the laptop
Zenodo	An archived copy of the release with its own DOI, under one concept DOI for the project	GitHub–Zenodo integration, triggered by the GitHub release
Documentation site	User guide, these design documents, the changelog	GitHub Pages, rebuilt on each release
Research site and LinkedIn	A short post on what the release adds and what it found	Written by Vijay in the write-up days of the month

Each release note lists the versions of every package inside it, including our own packages that attestqa depends on (bdd2pw, pw-emit, synthdata, pw-self-heal, sel2pw, usagecontract), so anyone can tell exactly what a release contains.

A DOI for every release gives each month's work a citable, permanent record. The concept DOI always resolves to the latest version; the version DOIs never change.

3. The pipeline

On every pull request and push to `main`: lint, type check, unit and rule tests, and the ledger replay tests over the Paper I corpus, on a Windows runner and a Linux runner. Required to pass before merge.

On a version tag:

1. Build every package and the desktop app.
2. Run the full test suite again, plus the app's end-to-end tests.
3. Sign the installer (Azure Trusted Signing, from CI, so the signing credential never sits on a laptop).
4. Publish changed npm packages with provenance.
5. Create the GitHub release with installer, checksums and SBOM.
6. Zenodo archives the release and mints its DOI.
7. Rebuild the documentation site.

If any step fails, nothing after it runs, and the release is fixed and re-tagged as the next patch version. A published version is never deleted or overwritten.

4. Release checklist (13th–15th of the month)

Step	Who
Freeze is clean: all tests green on both runners	CI, checked by Vijay
Changelog and release notes written (Changesets collects them during the month)	Claude drafts, Vijay edits
Version numbers set; release notes list package versions	Claude prepares
Tag `vX.Y.Z` pushed from Windows	Vijay
Pipeline watched to the end; installer downloaded and installed on a clean machine	Vijay
Zenodo DOI recorded in `CITATION.cff`, the master tracker and the research journal	Claude prepares, Vijay commits
Post written for the research site and LinkedIn	Claude drafts, Vijay edits and posts

Publishing, posting and tagging are Vijay's actions. Claude prepares everything up to the button.

5. Repository settings

Branch protection on `main` with required checks. Dependabot for dependency updates. CodeQL code scanning. GitHub secret scanning with push protection. Issue templates for bug reports, provider requests and new target sites. Discussions on, for questions. `SECURITY.md` explains how to report a vulnerability privately. `GOVERNANCE.md` states that pull

requests weakening a refusal rule are not merged without an argument backed by ledger evidence.

6. Research data

Study data is published separately from the software. For each study (S2 cross-model, S3 cross-site, S4 cross-type), the ledgers from public practice targets, the run manifest and the analysis scripts go to Zenodo as a dataset with its own DOI, after scrubbing. The hypotheses file and its hash are committed to the public repository before the study collects data, so the order of events is visible to anyone. Ledgers from private systems are never part of a dataset.

7. Our existing packages

bdd2pw, pw-emit, synthdata, pw-self-heal, sel2pw and usagecontract keep their own repositories, release cadences, npm pages and DOIs. When attestqa needs a change in one of them, the change is released there first and attestqa raises its version range in its next release. Their own users get the fix too, and their download and citation histories stay where they are.